[GLOSSARY] $ ./terms --list --all

واژه‌نامه

اصطلاحات
مهندسی داده

واژه‌نامه‌ی مصور ۵۲ مفهوم کلیدی، با تصویر، نمودار و انیمیشن.

ارائه‌دهنده

نیوشا بقایی

niyusha.baghayi@gmail.com

niyusha-baghayi

[PART 01] $ ./section --open

بخش ۱

مبانی و
پایپ‌لاین‌های داده

۱۲ اصطلاح: مهندسی داده، کلان‌داده، فرادادە، پایپ‌لاین داده، ورود داده و ...

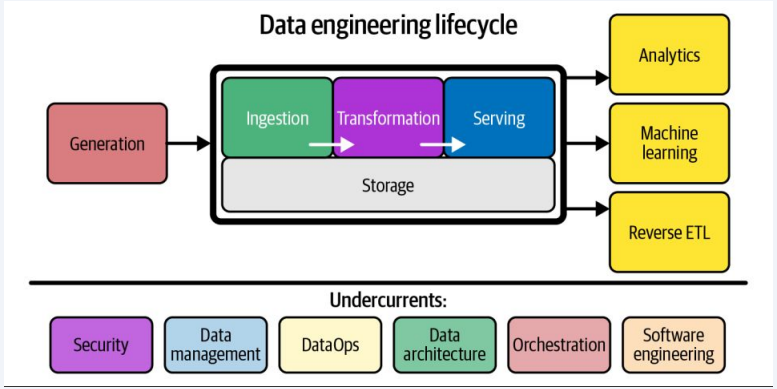
مهندسی داده

Data Engineering

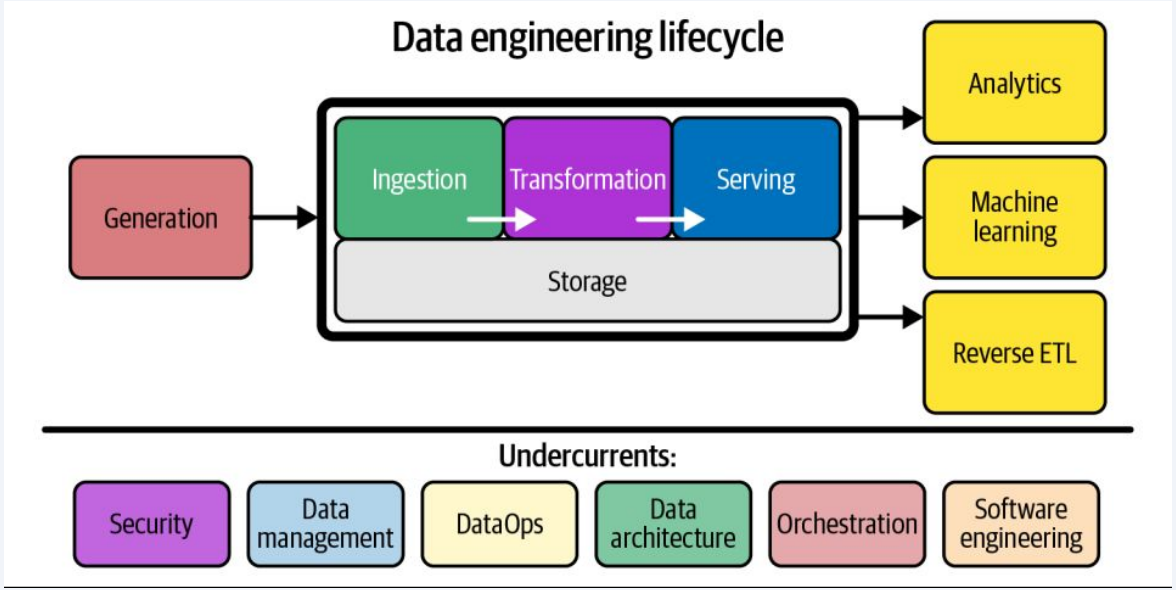
ساخت و **نگه‌داری** سیستم‌هایی که حجم زیادی از داده را مدیریت می‌کنند.

داده را **قابل‌اعتماد**، **مقیاس‌پذیر** و **آماده‌ی تحلیل** می‌کند.

کل **چرخه‌ی عمر داده** را پوشش می‌دهد: تولید، دریافت، تبدیل، ذخیره‌سازی و ارائه.



چرخه‌ی عمر مهندسی داده



کلان‌داده

Big Data

داده‌ای که ابزارهای معمولی از پس **ذخیره** یا **پردازش** آن برنمی‌آیند.

با **۶V** توصیف می‌شود: حجم، سرعت، تنوع، صحت، ارزش و تغییرپذیری.

لازم نیست **همه‌ی ۷ها** با هم باشند؛ حتی **یکی** از آن‌ها کافی است تا داده «کلان» حساب شود.



فراداده

Metadata

داده درباره‌ی داده: نام، نوع، اندازه، تاریخ ساخت و مالک.

مثال آشنا: روی یک فایل **راست‌کلیک** کنید و **Properties** را بزنید؛ آن‌چه می‌بینید فراداده است.

کمک می‌کند داده را **پیدا کنیم**، **بفهمیم** و **مدیریت کنیم**.

Properties · report.pdf	
نام	report.pdf
نوع	PDF
اندازه	۲٫۴ مگابایت
تاریخ ساخت	۱۴۰۳/۰۵/۱۲
مالک	نیوشا

این‌ها درباره‌ی فایل‌اند، نه محتوای فایل

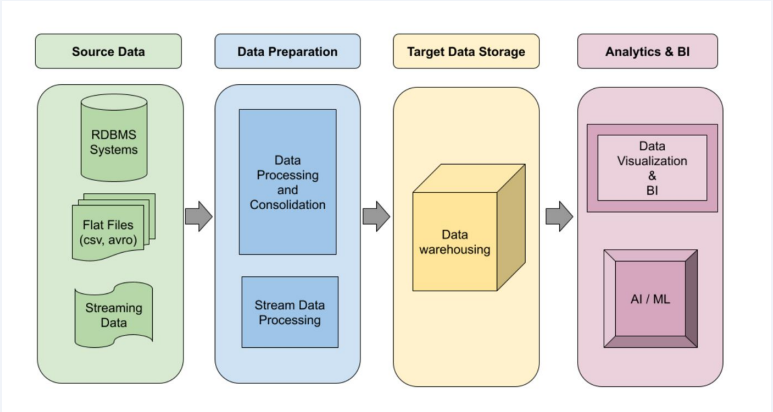
پایپ‌لاین داده

Data Pipeline

Pipeline یعنی **«لوله»**: داده مثل آب از یک سر وارد و از سر دیگر خارج می‌شود.

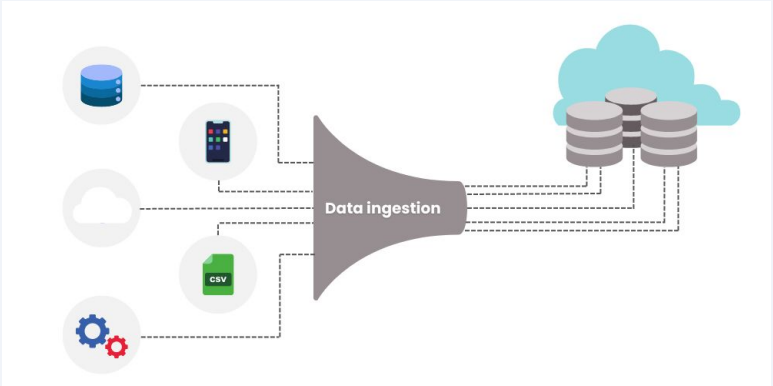
در مسیر از **ایستگاه‌هایی** مثل استخراج، تبدیل و بارگذاری می‌گذرد.

مثل لوله، **خودکار** و **پیوسته** کار می‌کند و خروجی هر مرحله، **ورودی مرحله‌ی بعد** است.



ورود داده

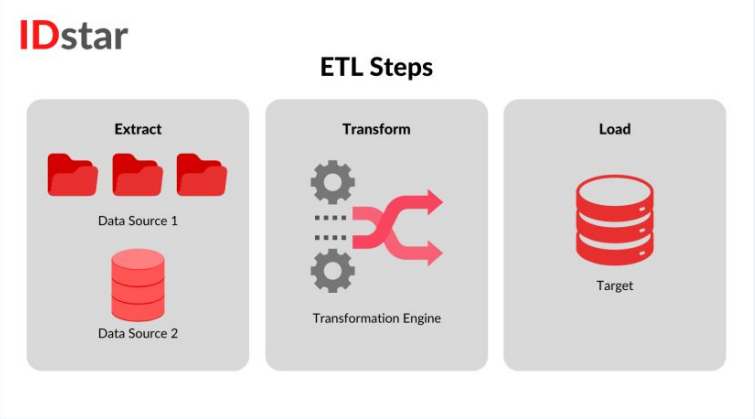
Data Ingestion



داده را از **منابع مختلف وارد** سیستم می‌کند.

به‌صورت **بلادرنگ (Stream)** یا **دسته‌ای (Batch)** انجام می‌شود.

اولین مرحله در پایپ‌لاین‌های داده است.



تبدیل داده

Data Transformation

داده را به **قالب قابل‌استفاده تبدیل** می‌کند.

پاک‌سازی، **قالب‌بندی** یا **محاسبات** را اعمال می‌کند.

داده را برای **تحلیل** یا **ذخیره‌سازی آماده** می‌کند.

ETL

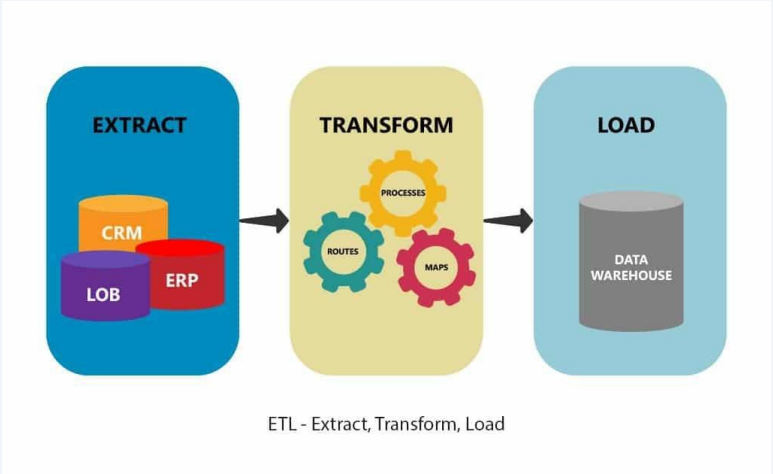
Extract, Transform, Load

داده را از منبع **استخراج** می‌کند (Extract).

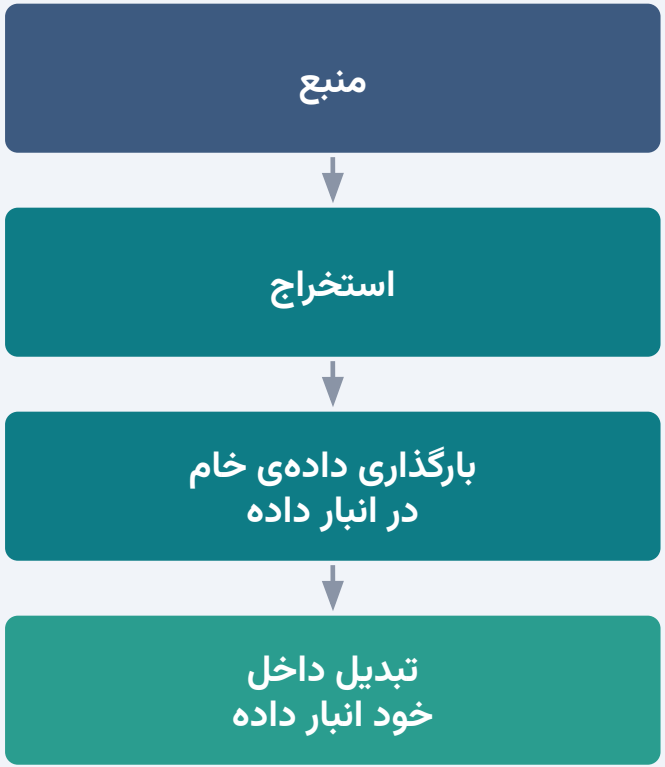
آن را به قالب مقصد **تبدیل** می‌کند (Transform).

آن را در سیستم ذخیره‌سازی، مثل انبار داده، **بارگذاری** می‌کند (Load).

مثال: هر شب سفارش‌های فروشگاه از پایگاه داده خوانده می‌شوند، مبلغ‌ها به تومان یکسان می‌شوند و نتیجه در انبار داده ذخیره می‌شود.



ETL - Extract, Transform, Load



ELT

Extract, Load, Transform

داده را **استخراج** و ابتدا **به‌صورت خام** در مقصد **بارگذاری** می‌کند.

تبدیل را بعداً، **داخل خود مقصد** (مثلاً انبار داده) انجام می‌دهد.

چون **داده‌ی خام حفظ می‌شود**، هر وقت لازم شد می‌توان تبدیل را عوض کرد و دوباره اجرا کرد.

ضبط تغییرات داده

CDC (Change Data Capture)

به‌جای کپی **کل جدول**، فقط **تغییرات** (درج، به‌روزرسانی، حذف) را منتقل می‌کند.

تغییرات را از **لاگ تراکنش‌های** پایگاه داده می‌خواند و **فشار کمی** به منبع می‌آورد.

مقصد **تقریباً بلادرنگ** به‌روز می‌ماند و **حتی حذف‌ها** را هم می‌بیند.



اجرای تکرارپذیر

Idempotency

اگر پایپ‌لاین را **دوباره اجرا** کنیم، **همان نتیجه** را می‌گیریم.

اجرای مجدد بعد از خطا **امن** است و **داده‌ی تکراری** ساخته نمی‌شود.



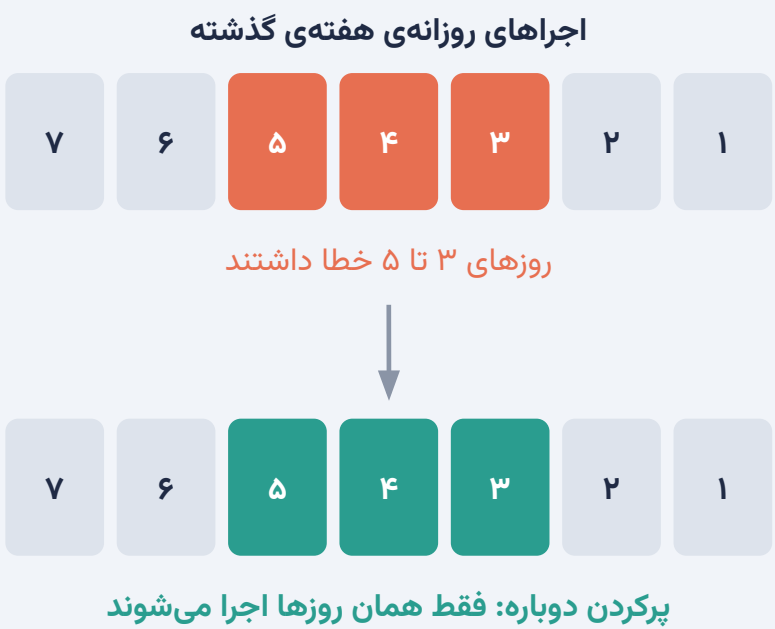
پرکردن دوباره

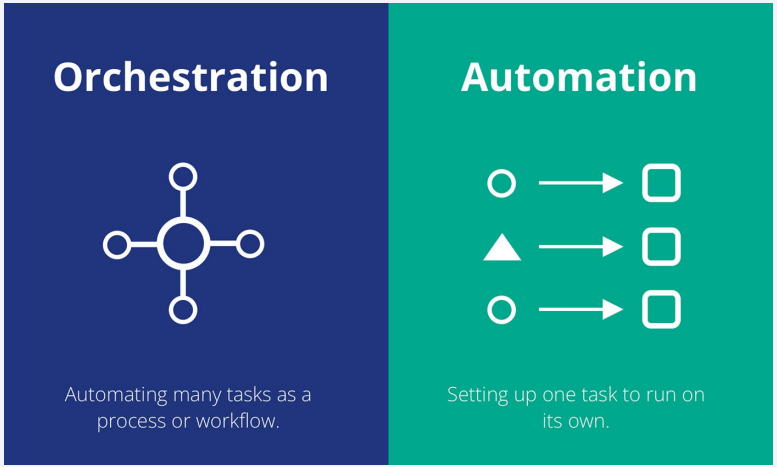
Backfill

اجرای پایپ‌لاین برای **تاریخ‌های گذشته**.

داده‌های تاریخی **گم‌شده** یا **نادرست** را پر می‌کند.

بعد از **رفع باگ** یا اضافه شدن **ستون جدید** لازم می‌شود.





هماهنگ‌سازی (ارکستراسیون)

Orchestration

کارها را بین سیستم‌ها **خودکار** و **هماهنگ** می‌کند.

کارها را به **ترتیب درست** اجرا می‌کند (**DAG**).

ابزارهای رایج: **Dagster**، **Airflow** و **Kubernetes**.

[PART 02] $ ./section --open

بخش ۰۲

پردازش و پیام‌رسانی

۸ اصطلاح: پردازش دسته‌ای، پردازش جریانی، معماری Lambda و Kappa، واسط پیام، مقصد داده و ...

پردازش دسته‌ای

Batch Processing

حجم زیادی از داده را **یک‌جا** پردازش می‌کند.

برای داده‌هایی که **فوریت زمانی ندارند** استفاده می‌شود.

مناسب کارهای **زمان‌بندی‌شده**، مثل **گزارش‌های روزانه**.



پردازش جریانی

Stream Processing

داده را **همان لحظه‌ی رسیدن** و **بلادرنگ** پردازش می‌کند.

مناسب کارهای **حساس به زمان**، مثل **تشخیص تقلب**.

داده را **به‌طور پیوسته** پردازش می‌کند.



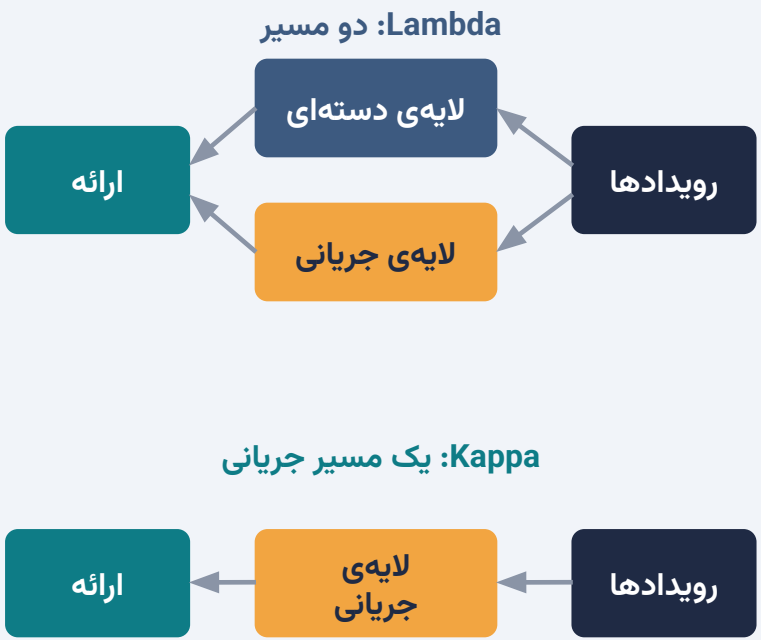
معماری Lambda و Kappa

Lambda and Kappa Architecture

Lambda: دو لایه دارد؛ **لایه‌ی دسته‌ای** (دقیق) و **لایه‌ی جریانی** (سریع).

Kappa: فقط **لایه‌ی جریانی** دارد، برای همه‌چیز.

Kappa ساده‌تر است: برای پردازش دوباره، **لاگ رویدادها از اول پخش** می‌شود.



استریمینگ

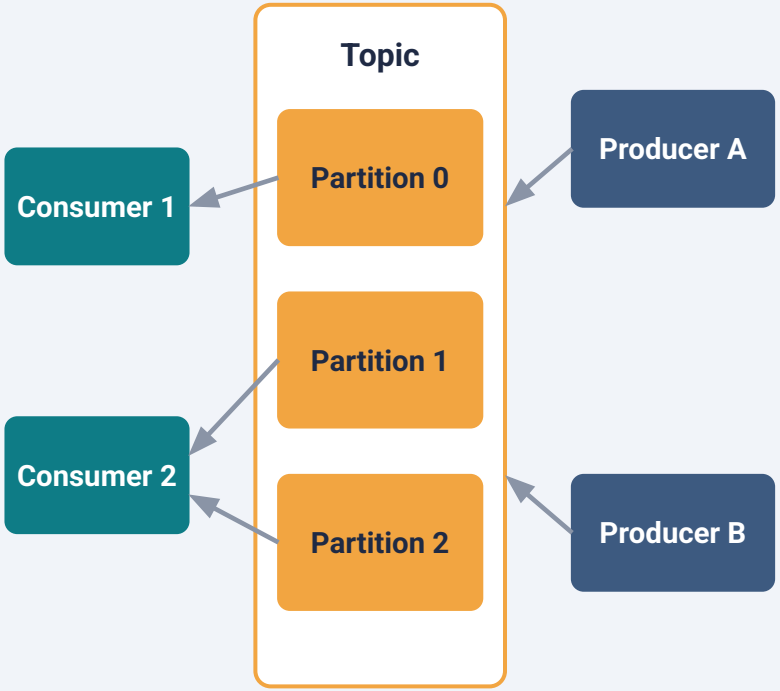
واسط پیام

Message Broker

پیام‌ها را بین سیستم‌ها جابه‌جا می‌کند؛ مثل **Kafka** و RabbitMQ.

Producer پیام را در **Topic** می‌نویسد و **Consumer** آن را می‌خواند.

سیستم‌ها را از هم **مستقل** می‌کند تا با **سرعت‌های متفاوت** کار کنند.



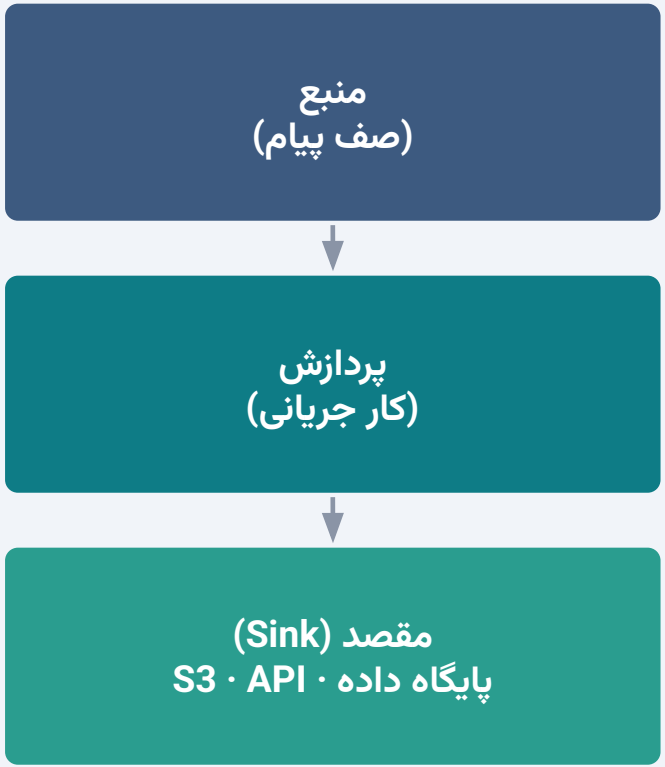
مقصد داده

Sink

مقصد نهایی که داده به آن فرستاده یا در آن ذخیره می‌شود.

نقطه‌ی مقابل **منبع (Source)** است: داده از منبع می‌آید و به Sink می‌رسد.

نمونه‌ها: **پایگاه داده**، **فضای ذخیره‌سازی** و **API**.



تضمین تحویل پیام

Delivery Semantics

حداکثر یک‌بار: تکرار ندارد، اما **ممکن است داده گم شود**.

حداقل یک‌بار: داده گم نمی‌شود، اما **تکرار** ممکن است.

دقیقاً یک‌بار: هر پیام **یک‌بار** حساب می‌شود؛ پیاده‌سازی سخت‌تر است.

At-most-once
۰ یا ۱ بار · ممکن است داده گم شود

At-least-once
۱ بار یا بیشتر · ممکن است تکرار شود

Exactly-once
دقیقاً ۱ بار · سخت‌ترین پیاده‌سازی

آفست

Offset

موقعیت در یک **جریان داده** (مثلاً Kafka) را مشخص می‌کند.

نشان می‌دهد پردازش **آخرین بار کجا متوقف شده** است.

به سیستم اجازه می‌دهد پس از **توقف** یا **خطا ادامه دهد**.



نقطه‌ی بازیابی

Checkpoint

عکسی از وضعیت یک فرایند در یک لحظه‌ی مشخص که ذخیره می‌شود.

پس از **خطا**، سیستم از **آخرین نقطه‌ی بازیابی** ادامه می‌دهد، نه از اول.

در **پردازش جریانی** برای **تحمل خطا** استفاده می‌شود.

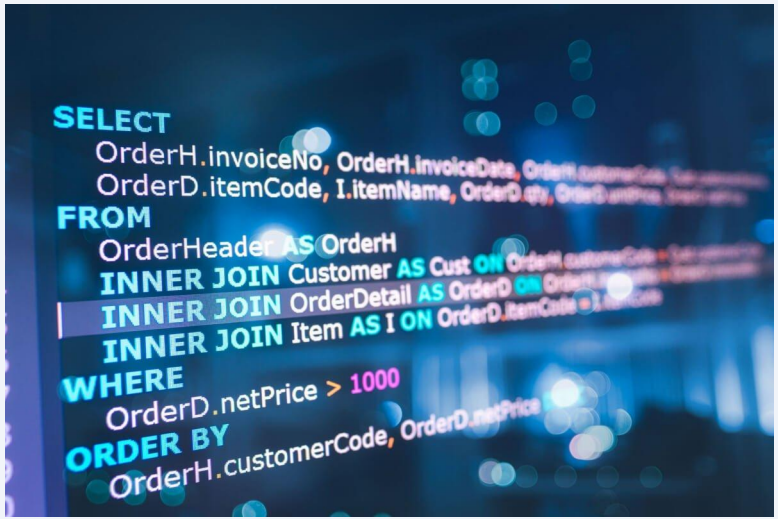


[PART 03] $ ./section --open

بخش ۰۳

پایگاه داده،
ذخیره‌سازی و مدل‌سازی

۱۶ اصطلاح: SQL، پایگاه داده‌ی رابطه‌ای، پایگاه داده‌ی NoSQL، ACID در برابر BASE، انبار داده و ...



SQL

Structured Query Language

زبان استاندارد برای **دسترسی** و **کار با** پایگاه‌های داده.

امکان **جست‌وجو**، **درج** و **به‌روزرسانی** داده را می‌دهد.

به‌طور گسترده در مدیریت داده استفاده می‌شود.

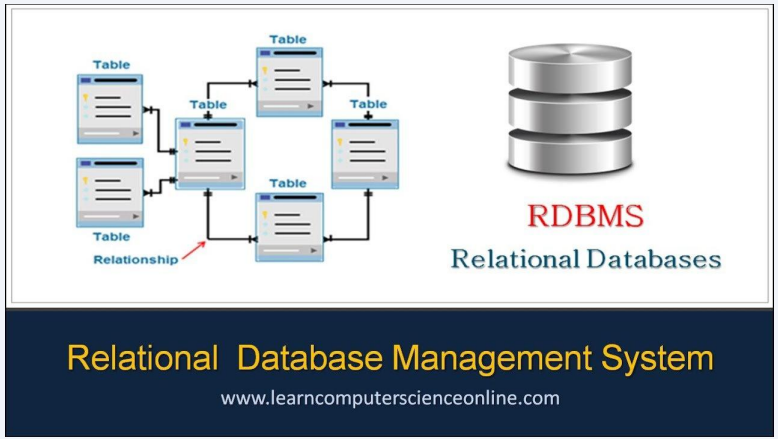
پایگاه داده‌ی رابطه‌ای

Relational Databases

داده را در **جدول‌های ساختاریافته** با سطر و ستون ذخیره می‌کند.

برای مدیریت و **پرس‌وجوی** داده از **SQL** استفاده می‌کند.

روابط را با **کلیدها** تعریف می‌کند؛ بهترین گزینه برای داده‌ی ساختاریافته.



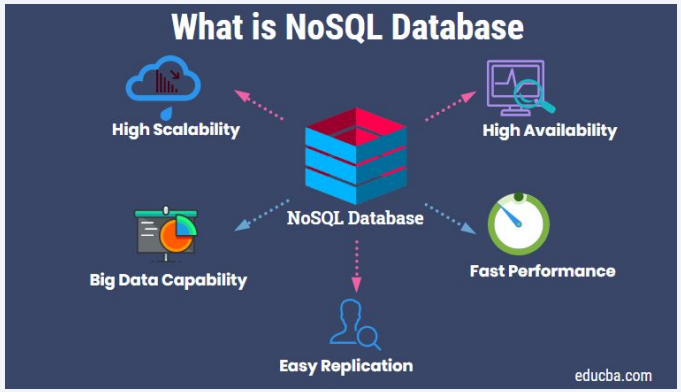
پایگاه داده‌ی NoSQL

NoSQL Databases

یعنی «**نه فقط SQL**»؛ پایگاه داده‌ای **انعطاف‌پذیر** و **غیررابطه‌ای**.

داده‌ی **بدون‌ساختار** یا **نیمه‌ساختاریافته** را مدیریت می‌کند.

برای نیازهای **کلان‌داده به‌راحتی مقیاس‌پذیر** است.




```
{
  "timestamp": "2024-10-22T15:35:10Z",
  "level": "INFO",
  "message": "User logged in successfully",
  "user_id": "12345"
}
```

ACID در برابر BASE

ACID vs BASE

ACID: تراکنش یا کامل انجام می‌شود یا اصلاً انجام نمی‌شود؛ **دقیق و امن**.

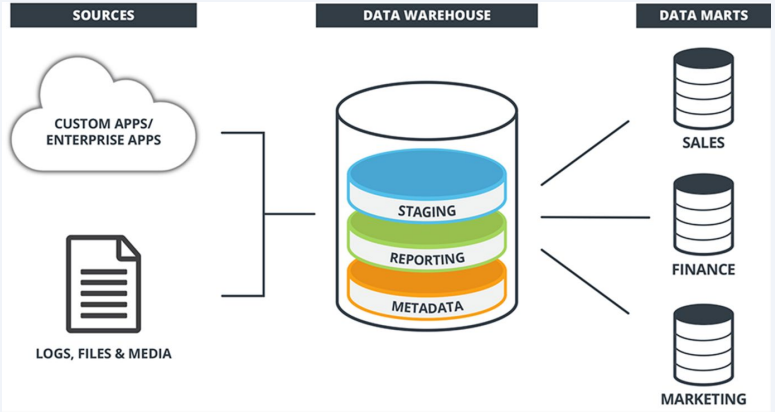
BASE: همیشه در دسترس است و داده **در نهایت** یکسان می‌شود؛ **منعطف و سریع**.

پایگاه‌های رابطه‌ای **ACID** و بسیاری از NoSQLها **BASE** را ترجیح می‌دهند.



انبار داده

Data Warehouse



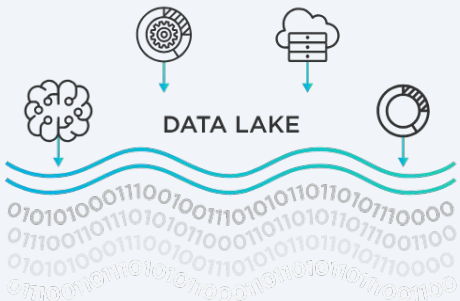
داده‌ی **ساختاریافته** و پاک‌سازی‌شده را از منابع مختلف **یک‌جا** جمع می‌کند.

یک **«منبع واحد حقیقت»** (Single Source of Truth) برای کل سازمان می‌سازد.

ساختار (Schema) **از قبل مشخص** است و برای **گزارش و تحلیل سریع** بهینه شده.

دریاچه‌ی داده

Data Lake



برخلاف انبار داده، داده را **خام** و در **هر قالبی** نگه می‌دارد.

ساختار **هنگام خواندن** مشخص می‌شود (Schema-on-Read).

ارزان و مناسب داده‌ی بدون‌ساختار و یادگیری ماشین؛ بدون مدیریت، به **«باتلاق داده»** تبدیل می‌شود.

لیک‌هاوس داده

Data Lakehouse

مزایای دریاچه‌ی داده و انبار داده را **ترکیب می‌کند**.

سازمان‌دهی، جست‌وجو و تحلیل **همه‌ی انواع داده** را ممکن می‌کند.



ذخیره‌سازی سطری و ستونی

Row vs Columnar Storage

سطری: مقادیر هر **سطر** پشت سر هم ذخیره می‌شوند؛ برای خواندن **یک رکورد کامل** سریع است.

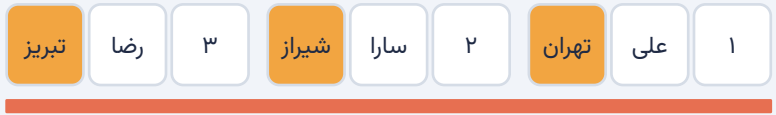
ستونی: مقادیر هر **ستون** پشت سر هم ذخیره می‌شوند؛ کوئری «همه‌ی شهرها» فقط **همان تکه** را می‌خواند.

در ستونی مقادیر مشابه کنار هم‌اند و **بهتر فشرده** می‌شوند؛ مناسب **تحلیل**.

کوئری: همه‌ی شهرها را بده

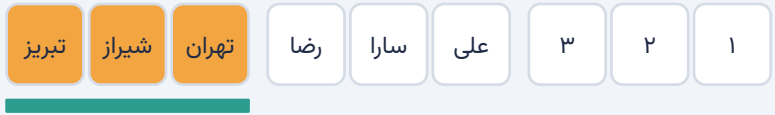
شناسه	نام	شهر
۱	علی	تهران
۲	سارا	شیراز
۳	رضا	تبریز

روی دیسک، سطری



باید همه‌چیز خوانده شود

روی دیسک، ستونی



فقط همین تکه

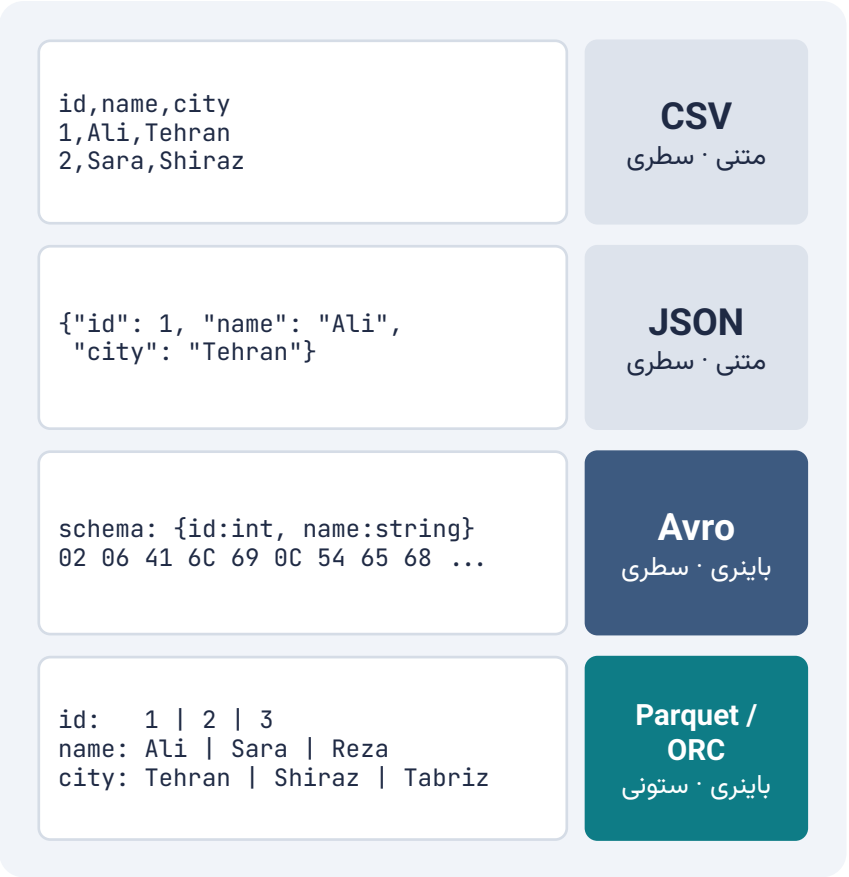
فرمت‌های فایل

File Formats

CSV / JSON: متنی و **خوانا**؛ مثلاً خروجی اکسل یا پاسخ یک API. ولی **حجیم** و کند است.

Avro: باینری و **سطری**، با Schema همراه فایل؛ مثلاً پیام‌هایی که در **Kafka** جابه‌جا می‌شوند.

Parquet / ORC: باینری و **ستونی**؛ مثلاً جدول‌های بزرگ **دریاچه‌ی داده** که روی آن‌ها تحلیل می‌کنیم.

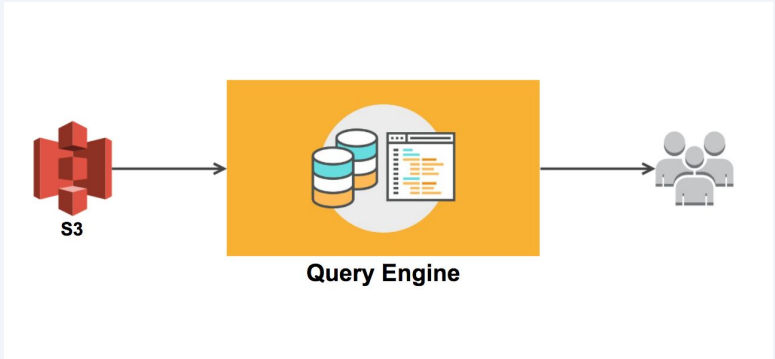


موتور کوئری

Query Engine

نرم‌افزاری که روی **پایگاه داده** یا **فضای ذخیره‌سازی** قرار می‌گیرد.

کوئری‌ها را اجرا و **پاسخ** را به کاربران یا برنامه‌ها برمی‌گرداند.



نرمال‌سازی و غیرنرمال‌سازی

Normalization vs Denormalization

نرمال‌سازی: هر اطلاعات **فقط یک‌بار** ذخیره می‌شود؛ جدول‌ها با **کلید** به هم وصل‌اند.

مثال: اگر شهر علی عوض شود، در غیرنرمال باید **همه‌ی سفارش‌هایش** را اصلاح کنیم؛ در نرمال فقط **یک جا**.

غیرنرمال‌سازی: تکرار **عمدی** داده تا کوئری **بدون Join** سریع شود؛ مناسب **تحلیل**.



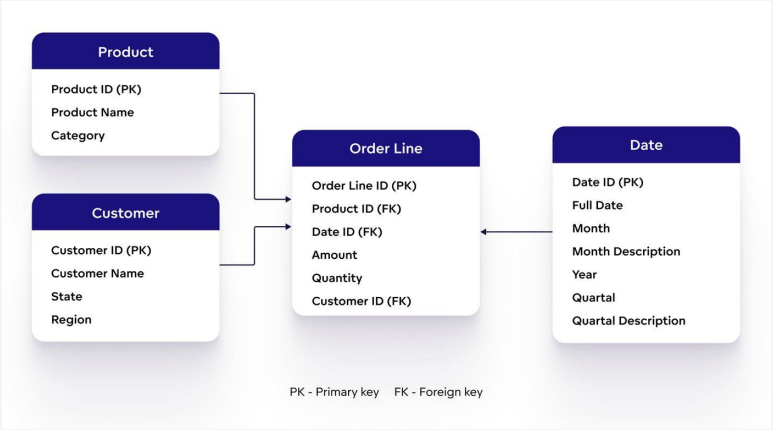
مدل‌سازی داده

Data Modeling

ساختار داده را **طراحی** می‌کند.

روابط بین داده‌ها را **تعریف** می‌کند.

داده را برای **استفاده** و **درک بهتر** سازمان‌دهی می‌کند.



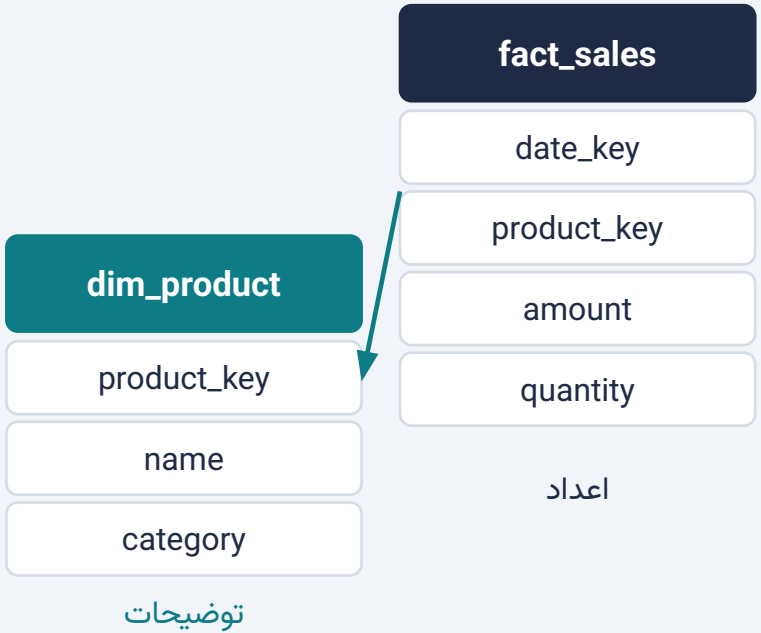
جدول Fact و Dimension

Fact and Dimension Tables

جدول Fact: **رویدادهای** قابل‌اندازه‌گیری، مثل مبلغ هر فروش.

جدول Dimension: **زمینه**؛ چه کسی، چه چیزی، کجا و چه زمانی.

Factها **عدد** نگه می‌دارند و Dimensionها **توضیح**.



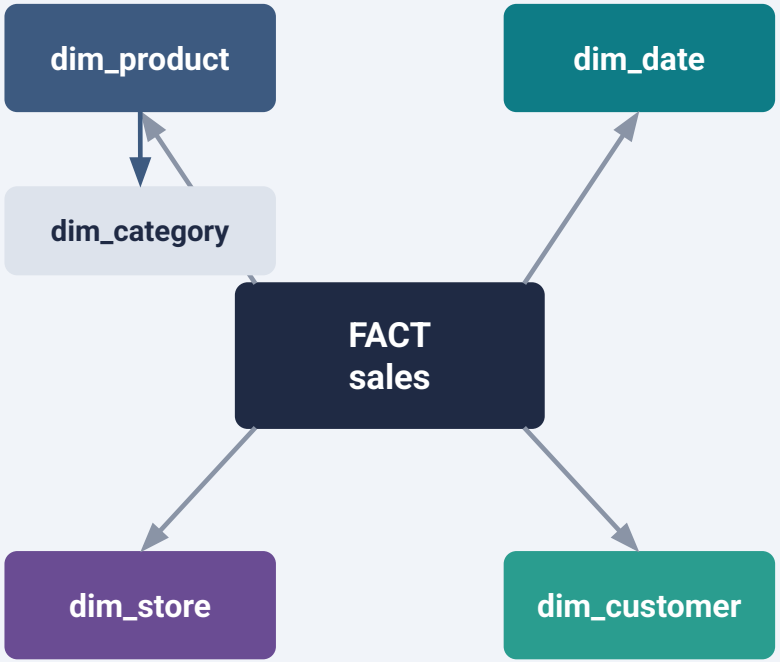
اسکیمای ستاره‌ای و دانه‌برفی

Star and Snowflake Schema

ستاره‌ای: یک **جدول Fact** متصل به **جدول‌های Dimension**.

دانه‌برفی: Dimensionها به **جدول‌های بیشتری** شکسته می‌شوند.

ستاره‌ای **ساده‌تر و سریع‌تر** است؛ دانه‌برفی **فضای کمتری** می‌گیرد.



dim_category بخش دانه‌برفی است

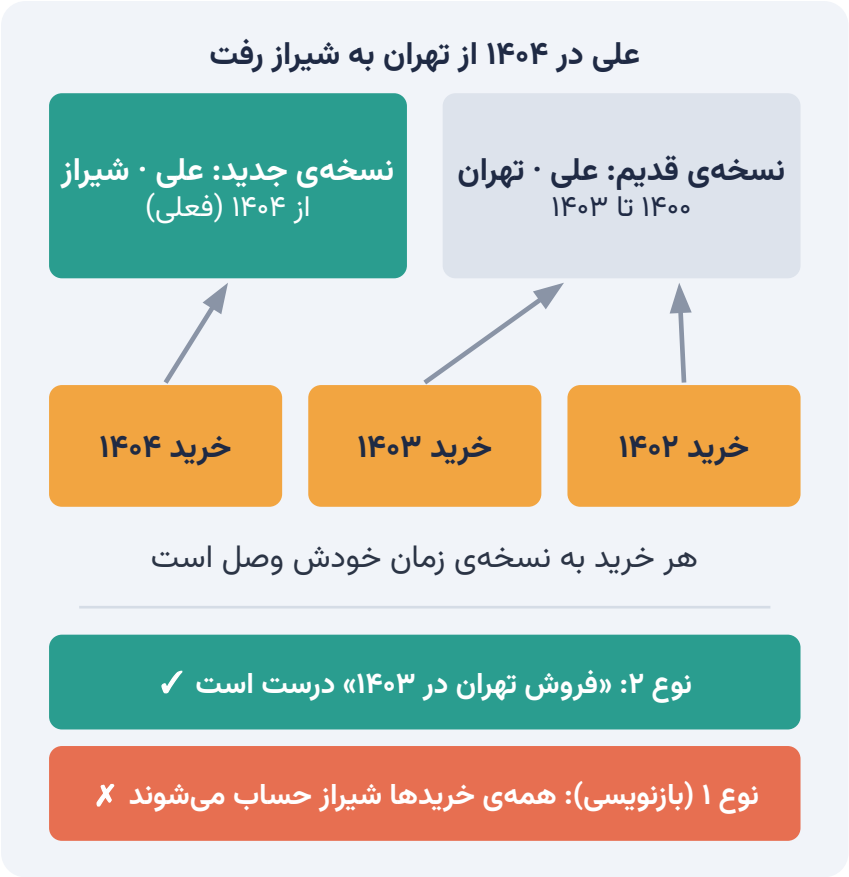
ابعاد با تغییر آهسته

SCD (Slowly Changing Dimension)

وقتی مشخصات یک **مشتری یا محصول** عوض می‌شود (مثلاً شهر)، **گزارش‌های گذشته** نباید غلط شوند.

نوع ۱: مقدار قبلی **بازنویسی** می‌شود؛ ساده است، ولی **تاریخچه از بین می‌رود**.

نوع ۲: یک **نسخه‌ی جدید** ساخته می‌شود و هر خرید به **نسخه‌ی زمان خودش** وصل می‌ماند؛ رایج‌ترین روش.



دیتامارت

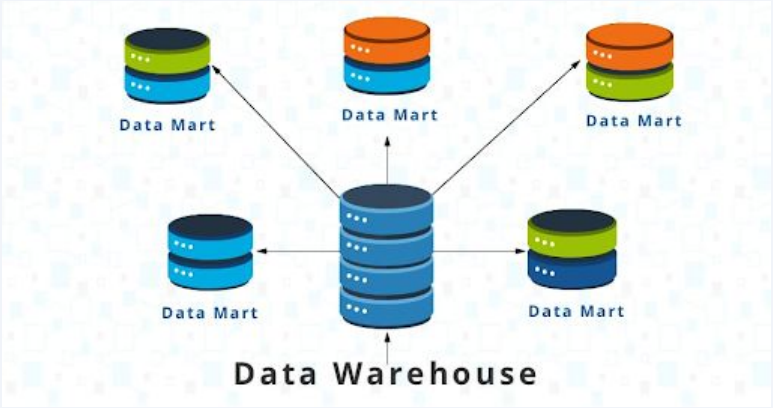
Data Mart

بخشی **کوچک‌تر** و **موضوعی** از انبار داده.

به یک **حوزه‌ی کسب‌وکار** خدمت می‌کند، مثل **فروش** یا **بازاریابی**.

دسترسی سریع‌تر به **داده‌ی مرتبط** را برای یک تیم فراهم می‌کند.

مثال: تیم فروش فقط جدول‌های فروش روزانه‌ی هر شعبه را می‌بیند، نه کل انبار داده را.



[PART 04] $ ./section --open

بخش ۰۴

سیستم‌های توزیع‌شده، کیفیت و حاکمیت داده

۱۶ اصطلاح: سیستم‌های توزیع‌شده، کلاستر، مقیاس‌پذیری، تکثیر داده، پارتیشن‌بندی داده و ...

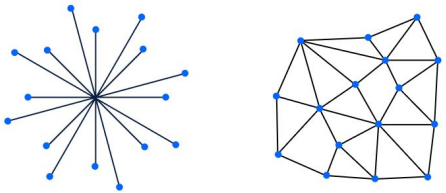
سیستم‌های توزیع‌شده

Distributed Systems

سیستم‌هایی که روی **چند ماشین** اجرا می‌شوند.

بار کاری را تقسیم می‌کنند تا **کارایی** بهتر شود.

قابلیت اطمینان و **تحمل خطا** را افزایش می‌دهند.



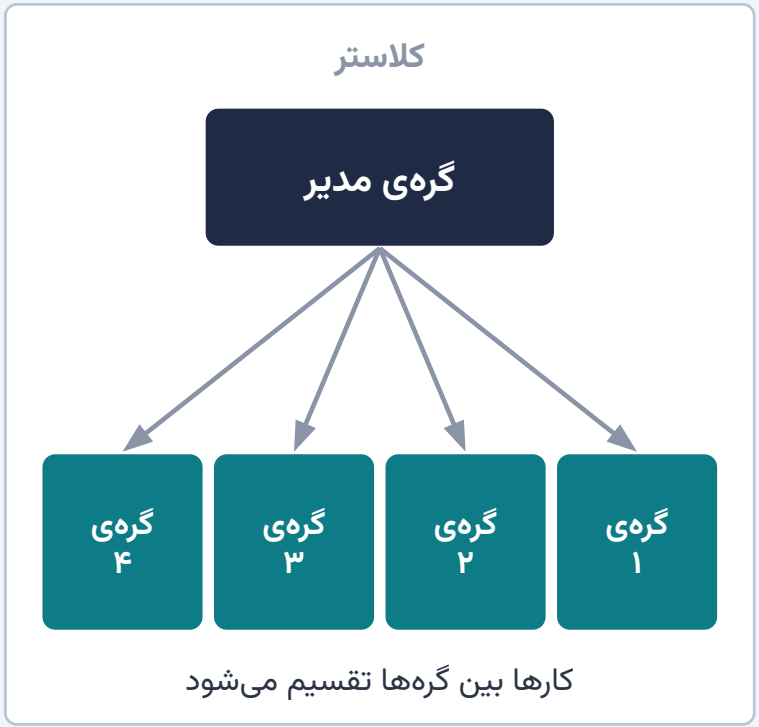
کلاستر

Cluster

گروهی از سرورها (**گره‌ها**) که مثل **یک سیستم واحد** با هم کار می‌کنند.

یک **گره‌ی مدیر** کارها را بین **گره‌های کارگر** تقسیم می‌کند.

اگر یک گره از کار بیفتد، **بقیه** کار را ادامه می‌دهند.

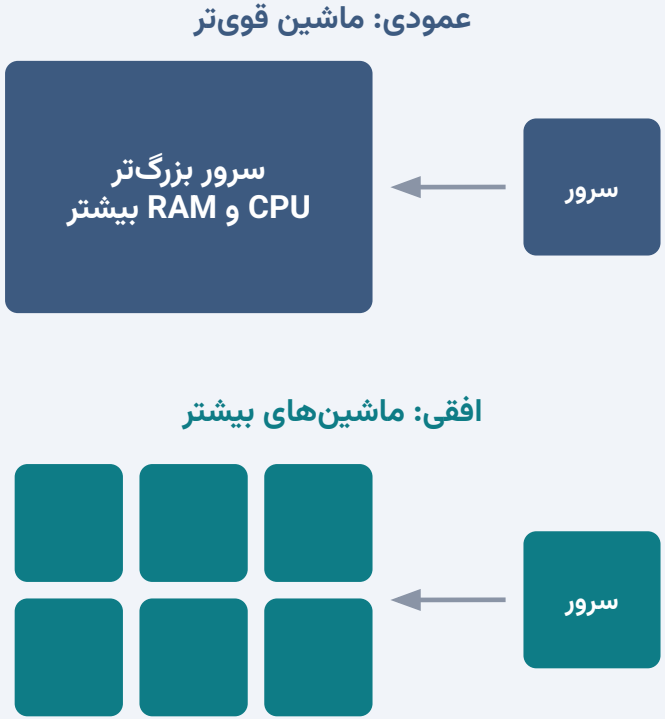


مقیاس‌پذیری
Scalability

توانایی سیستم برای **رشد** همراه با افزایش **داده و کاربران**.

عمودی: ماشین قوی‌تر. **افقی**: ماشین‌های بیشتر.

در کنار **قابلیت اطمینان** (Reliability) و **نگهداشت‌پذیری** (Maintainability)، سه ویژگی کلیدی یک سیستم داده‌ی خوب است.



تکثیر داده

Data Replication

داده را در **چند مکان کپی** می‌کند.

دسترس‌پذیری داده را حفظ و **افزونگی** ایجاد می‌کند.

در برابر **از دست رفتن داده** محافظت می‌کند.



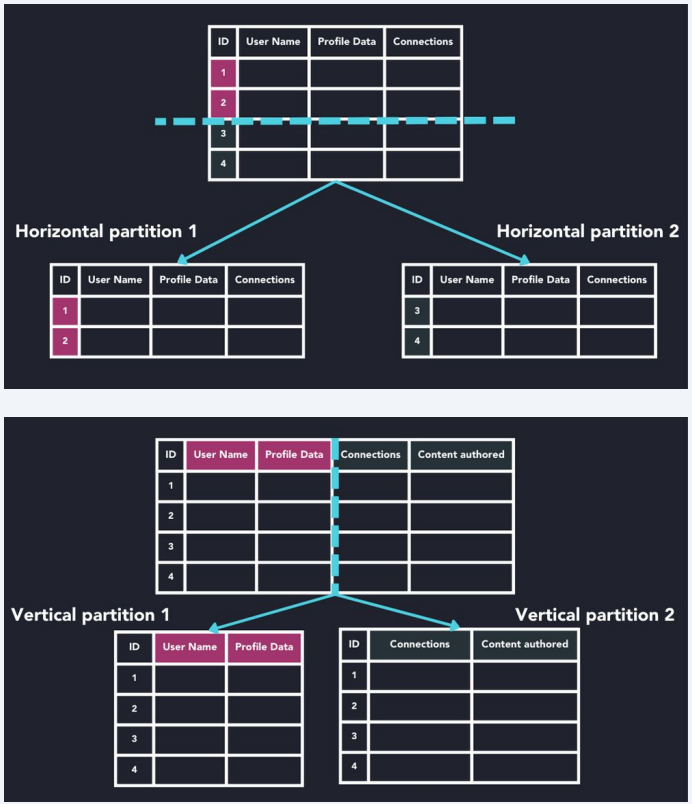
پارتیشن‌بندی داده

Data Partitioning

یک جدول بزرگ را بر اساس یک ستون (مثلاً **تاریخ**) به **بخش‌های کوچک‌تر** تقسیم می‌کند.

کوئری **سریع‌تر** می‌شود: فقط **پارتیشن‌های لازم** خوانده می‌شوند و پارتیشن‌ها می‌توانند **موازی** پردازش شوند.

مدیریت ساده‌تر: مثلاً داده‌ی یک ماه قدیمی را با **حذف یک پارتیشن** پاک می‌کنیم.



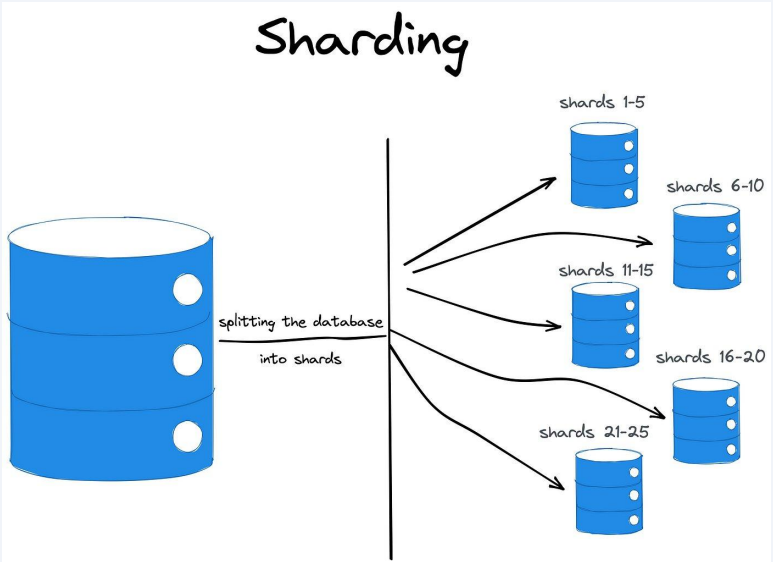
شاردینگ داده

Data Sharding

یک پایگاه داده‌ی بزرگ را به **بخش‌های مستقل** (Shard) **تقسیم** می‌کند.

هر Shard روی یک **سرور جداگانه** قرار می‌گیرد.

بار را پخش و **کارایی** را بهتر می‌کند.



قضیه‌ی CAP

CAP Theorem

یک سیستم توزیع‌شده نمی‌تواند **هر سه** را کامل تضمین کند.

سازگاری، **دسترس‌پذیری** و **تحمل پارتیشن** (C, A, P).

هنگام قطعی شبکه باید بین **سازگاری** و **دسترس‌پذیری** انتخاب کرد.



سازگاری داده

Data Consistency

یعنی **همه‌ی نسخه‌های** داده در یک لحظه **یک مقدار** را نشان دهند.

به‌خاطر **تکثیر داده**، نسخه‌ها ممکن است **مدتی با هم فرق** کنند.

همان **C** در قضیه‌ی **CAP** است؛ «سازگاری نهایی» در **BASE** یعنی نسخه‌ها **در نهایت** یکسان می‌شوند.



حاکمیت داده

Data Governance

حاکمیت داده چتر بالای همه است: **قوانین، نقش‌ها و مسئولیت‌ها** برای مدیریت داده.

کیفیت، کاتالوگ، شجره‌نامه و نگهداری **زیرمجموعه‌ی** آن‌اند: یا **هدف** حاکمیت‌اند، یا **ابزار و سیاست** اجرای آن.

مثل کتابخانه: حاکمیت **مقررات و مدیریت** آن است و کاتالوگ **سیستم جست‌وجوی** کتاب‌ها.



کیفیت داده

Data Quality

کیفیت داده یعنی داده **برای کاری که می‌خواهیم** قابل‌اعتماد باشد.

با **شش بُعد** سنجیده می‌شود؛ مثلاً **معتبر** بودن (قالب درست) با **دقیق** بودن (مطابق واقعیت) فرق دارد.

این بررسی‌ها **خودکار** و داخل پایپ‌لاین انجام می‌شوند تا داده‌ی خراب به **داشبورد** نرسد.



شش بُعد کیفیت داده، با مثال از جدول سفارش‌ها

یکپارچگی (سلامت) داده

Data Integrity

داده **دقیق** و **قابل‌اعتماد** است.

از **خرابی** یا **تغییرات غیرمجاز** جلوگیری می‌کند.

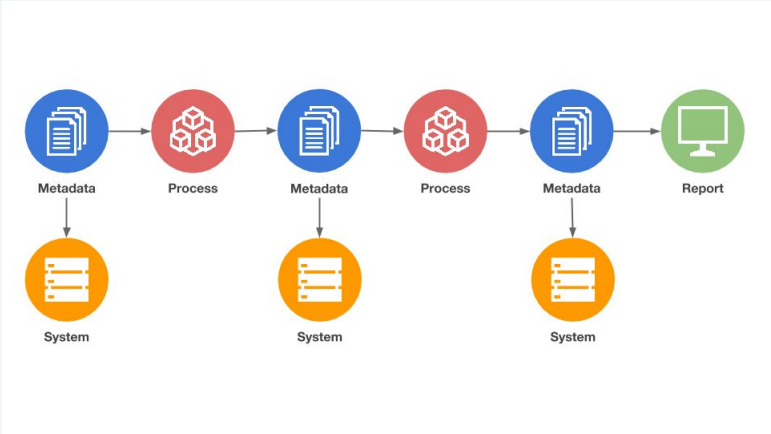
اعتماد به داده را در همه‌ی سیستم‌ها حفظ می‌کند.

سفارش شماره‌ی ۱۰۲۴

- ✓ شناسه یکتاست
- ✓ مبلغ خالی نیست
- ✓ تاریخ معتبر است
- ✓ فقط تغییرات مجاز

شجره‌نامه‌ی داده

Data Lineage



مثل یک **شجره‌نامه** نشان می‌دهد داده **از کجا آمده** و **به کجا رسیده** است.

مشخص می‌کند داده **در هر مرحله چه تغییری** کرده است.

برای **عیب‌یابی** و **ممیزی (Audit)** ضروری است.

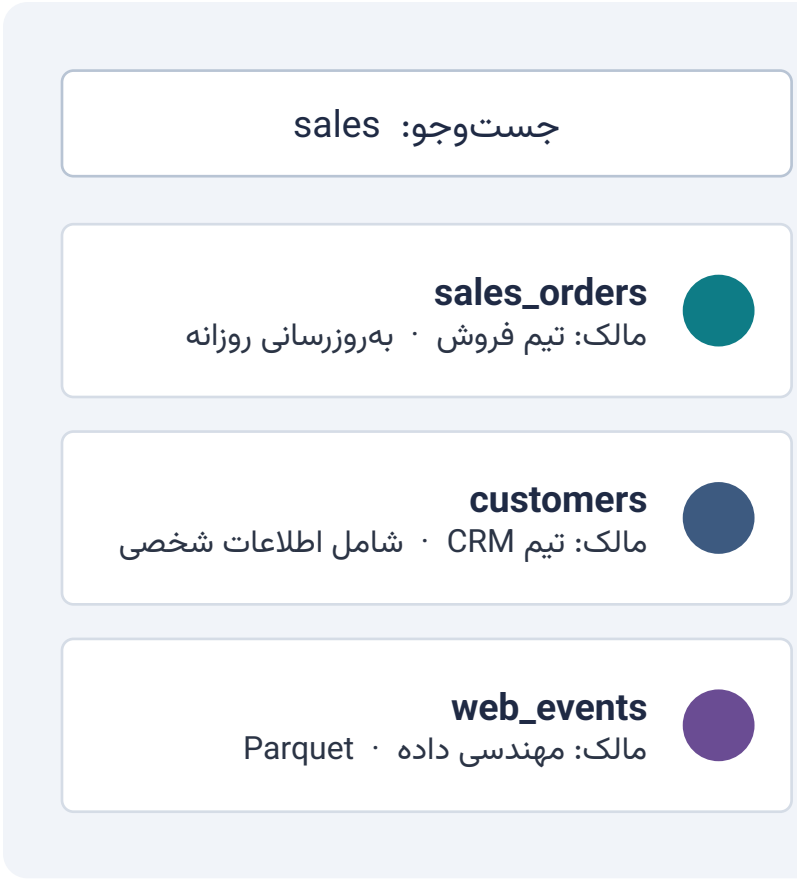
کاتالوگ داده

Data Catalog

فهرستی قابل‌جست‌وجو از همه‌ی دارایی‌های داده.

توضیحات، مالک، ساختار (Schema) و **شجره‌نامه‌ی** داده را نشان می‌دهد.

مثل **کاتالوگ کتابخانه:** بدون آن، پیدا کردن داده‌ی درست سخت است.





نگهداری داده

Data Retention

تعیین می‌کند داده **چه مدت** نگه‌داری شود.

به **رعایت قوانین** و **مدیریت فضای ذخیره‌سازی** کمک می‌کند.

معمولاً با **حذف خودکار** پس از یک دوره‌ی مشخص همراه است.

عملیات

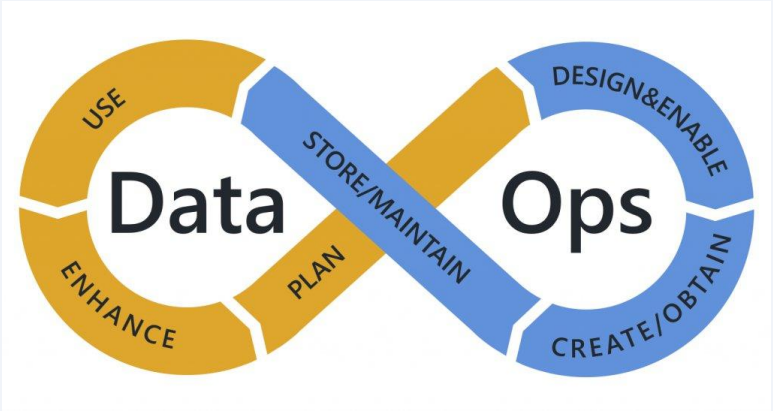
DataOps

DataOps

پایپ‌لاین‌های داده را برای کارایی بیشتر **خودکار** می‌کند.

با **بررسی‌های مداوم کیفیت داده** را تضمین می‌کند.

همکاری تیم‌ها را بهتر و پایپ‌لاین‌ها را **پایش** می‌کند.



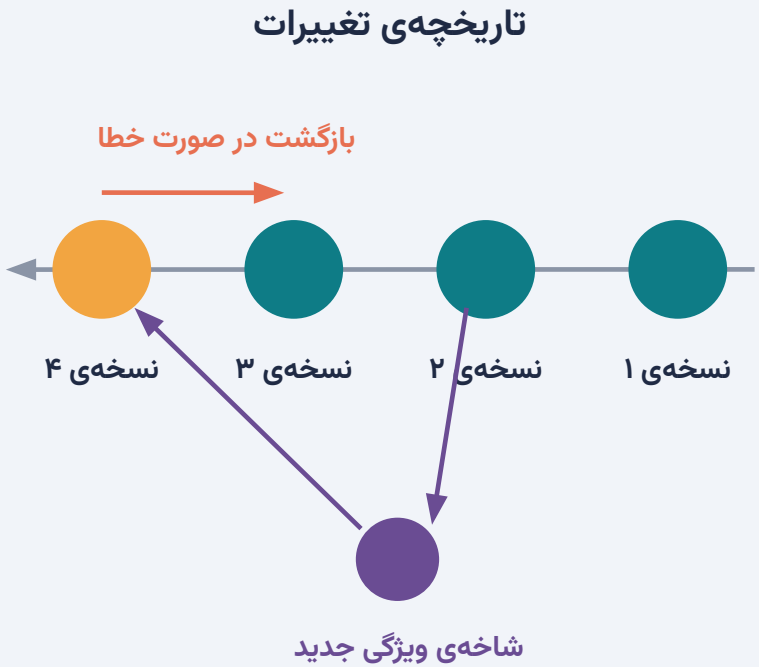
کنترل نسخه

Version Control

تغییرات کد یا فایل‌ها را در طول زمان **ثبت می‌کند**.

همکاری بین اعضای تیم را ممکن می‌کند.

تاریخچه را نگه می‌دارد تا در صورت نیاز **بازگشت** ممکن باشد.



[END] $ exit 0



سپاس از توجه شما
سؤالی دارید؟

niyusha.baghayi@gmail.com

niyusha-baghayi